Final Report

Ezra Klukas & Justin Qian

ENPH 353

December 6th, 2025

Table of Contents

Introduction

Background

This year's competition was a "autonomous robot detective" that drove around a course, reading clues on road signs to solve a "murder mystery." In effect, this presented us with two primary challenges: driving while obeying traffic rules and avoiding obstacles, and accurately recognizing characters on each clue board to solve the mystery, doing this without external input once the timer started.

Figure 1. Competition surface.

Our approach was to maximize driving speed through quick, artisanal image processing (mostly using binary pixel masks) based off a low-resolution camera feed, while employing high-resolution cameras angled towards each side to effectively read signs.

Responsibility Delegation

Justin was responsible for driving throughout the course, and Ezra was responsible for sign detection and reading. Ezra also explored driving on the final section of the course.

Description of the overarching software architecture

For plate detection, plate_detect.py processed images from the right and left camera feeds and published the appropriate messages to /score_tracker.

For driving, line_follow.py extracted features from the central camera feed and output to /drive_info, which drive.py used for its state machine to output /cmd_vel, also feeding back to line_follow.py any section changes through /B1/loc. drive_hill.py took in the camera feed and output a /cmd_vel for the final section of the course.

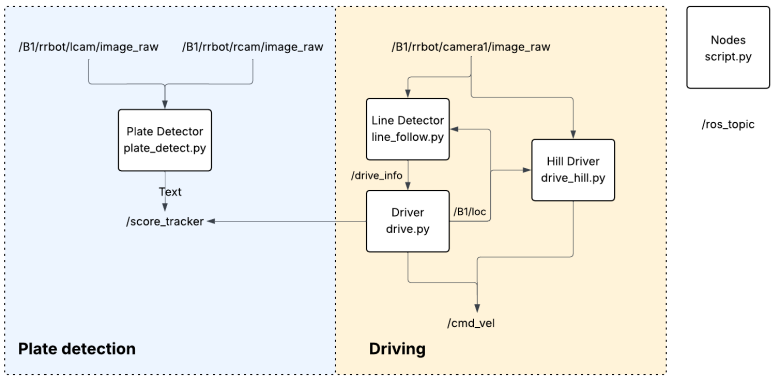

Figure 2: High-level diagram of our software architecture. Boxes represent ROS nodes and lines of text are ROS topics.

Discussion

Driving

High Level Overview

We employed a state machine with limited state space (i.e. no PID), with decisions made based on pixel masks and hardcoded logic from a 320x320 front facing camera feed, as well as a CNN with a discrete state space, for different sections of the course.

We divided the track into 4 sections: (i) standard gravel road before the first pink line, (ii) dirt track between the first two pink lines, (iii) offroad section, (iv) hill after the final pink line. For (i)-(iii), we used pixel masks for important driving related features (i.e road, pedestrian, truck, bridge, baby yoda), and for (iv) we used a 120k parameter CNN trained on the blue channel, bottom half of the camera feed.

We employed a pure state-machine approach with the original intention of maximizing speed, as each image could be processed and driving logic output in <10 ms, but apparent ROS constraints, as well as the need to get clear sign pictures, meant that this could not be fully exploited.

Sections (i)-(ii): line following pixel thresholding

We thresholded the line based on pixel values. In section (i) this was essentially purely the line given the strong contrast between line and pavement; in section (ii) there was significant noise in the road, which was reduced via a Gaussian blur, threshold, and morphological transformation to eliminate small patches of noise.

Figure 3. Driving masks. Left: line mask in section (i). Center: line mask in section (ii). Note the appearance of noise and the broken line. Right: water mask on the bridge.

Driving logic was determined by a state machine with 17 discrete states (including 6 'stop' states for different situations) with dozens of branches (49 if statements total), making it impossible to describe fully in this length-constrained report. A concise summary is as follows. When the line was in front of the robot (appearing in a preset 'box' of pixel coordinates), or appeared much closer on one side than the other, the robot would turn in the direction where the line appeared higher up (and was thus further; i.e. line appears lower on the left side --> turn right). It would stop turning when the line was an equal distance away on both sides (technically, when it approached that level, given the delay between image capture/processing and robot motion). Some hardcoding was used to deal with special driving edge cases (passing the crosswalk, entering and exiting the loop, handling NPCs). These were handled with time-locking states, so that for a certain length of time the robot wouldn't react to new camera information, to ensure its current action was completed. On the bridge new driving logic was used that only paid attention to the water, turning if the water was approaching on one side. After we passed the bridge (detected again with the water pixel mask), the robot was programmed to drive at the pink line, which at that point was visible and could be reached without an offroad penalty.

Line following was fairly consistent, though not all edge cases were fully resolved due to time constraints. Sometimes (~10% frequency) the robot would immediately turn left or right upon exiting the bridge and end up back in the water, and occasionally (~5%) it would mistake noise for line in section 2, and drive offroad as a result.

Section (iii): driving following Baby Yoda

We realized Baby Yoda directed us from the pink line to the tunnel, so we just waited until he appeared and then followed him, moving forward if he was in the center of the screen, turning if he was on one side of the screen and stopping if we were about to run into him. Eventually, the "Baby Yoda mask" pixels were also found on the tunnel and parked car such that the robot would conveniently not notice as Baby Yoda disappeared from view, at which point we transitioned to another state machine using the parked car to guide the robot to line up with the tunnel. Due to

time constraints, for competition we immediately teleport the robot to be perfectly aligned with the tunnel at the pink line, instead of risking poor alignment and becoming stuck in the tunnel.

Sections (i)-(iii): NPC detection

NPCs were detected by characteristic pixel values not seen to the same level elsewhere in the course. For the pedestrian, we noticed that their pants had consistent pixel values when viewed from any angle. We assumed the robot faced forward, so the pedestrian was declared 'in view' when their pants were in the middle of the screen (i.e. we disregarded edges), which would trigger the robot to stop at the crosswalk. If the pedestrian wasn't seen on the road, the robot would drive forward, accelerating fast enough to usually avoid collision if the pedestrian then started crossing.

For the truck, most of it had consistent grayscale values, which were thresholded to avoid also capturing the road or any other features. Since our robot was faster than the truck, we had to avoid crashing into it. If the truck was directly in front of us (defined at a pixel threshold), we stopped temporarily to allow the truck to go ahead. If the truck wasn't spotted, driving proceeded as normal. For Baby Yoda, we applied separate green and gray masks (green before he turned, gray after, since we see different parts of him and thus different pixel values) and followed him as discussed above.

Our performance on NPC detection was generally solid, though there were a few unresolved edge cases with this probably being our biggest weakness. Specifically: pedestrian detection was good but if the pedestrian started crossing at the same time as the robot, then the two occasionally collided. Truck detection was good within the loop, but the state machine didn't always let it get far enough ahead of the robot before it entered the loop; in competition, this specific edge case was exposed. Baby Yoda detection and tracking was perfect in testing.

Figure 4. NPC masks. Left: pedestrian pants masked in blue. Center: Truck masked in gray. Right: Yoda masked in green.

Section (iv): CNN

For driving on the final section, the constantly changing pixel values made the artisanal approach of previous sections challenging, so we opted for a lightweight CNN instead. Data was acquired from the camera (320x320), and preprocessed by taking the blue channel and roughly the bottom

half (pixel y-values 140-300) as that contained all necessary information for driving. Input size was thus shrunk by a factor of 6. 903 training images were collected, and after downsampling the FWD state (described below) and using a 90-10 train-val split we had 745 training and 83 validation images.

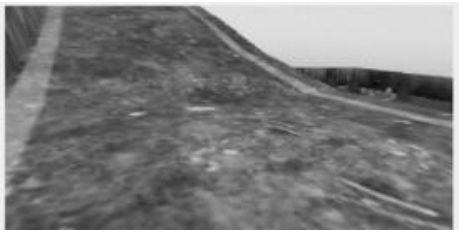

Figure 5. Sample CNN input image.

We used a CNN with 4 convolution layers, each one followed by max pooling, batch normalization, and dropout. Global average pooling was applied after the final convolution layer, then into a dense layer before our final layer with 5 states. The final layer used softmax activation, and all other layers used ReLU activation. This architecture was designed through iteration for the best performance using relatively few parameters. We used categorical cross-entropy loss, the AdamW optimizer with a learning rate of 2.5×10^{-3} (empirically determined), training for 100 epochs, and taking the model with the lowest validation loss. See appendix for model architecture summary and train/validation loss. Note that validation loss stayed comparatively high and occasionally spiked, though this was expected since many images did not have a clear best ground truth label. Mispredicted labels would upon inspection usually be reasonable actions given the input.

Data was labeled with only the ground truth state output, with the states FWD, FWD_LEFT, LEFT, RIGHT, and STOP, where the FWD_LEFT state drove the robot forward and left simultaneously. This combination of states was chosen for efficiency, given that only left turns were needed on the hill (the RIGHT state was only for correction in case the robot turned too far left). In terms of training data balance, the FWD state was overrepresented (54.9%) and the RIGHT (8.2%) and STOP (1.7%) states underrepresented, which was remedied by downsampling the FWD examples in training. We found that even despite the limited number of RIGHT and STOP examples, the model still learned appropriate behaviours for both. Our error analysis was watching the robot drive in sim: if it ever drove off at a certain location, we augmented the training set at that location. If it went offroad, we put more training images of it cutting fewer corners.

Plate Detection and Recognition

Implementation Overview

Our main priority in developing plate detection was for integration with driving to be passive, not impose constraints on driving, and be lightweight for a seamless integration. We used two additional cameras to ensure unobstructed plate image capture, regardless of the driving path. A mask was developed for the clue board borders, enabling fast and reliable plate detection. Inverse

perspective transforms were used to unskew the plates, centre and align the text, and variable masks and contour ordering was used to partition the image into discrete binary ordered characters for our CNN to inference, and the results were compiled into strings for the guess to be made.

Figure 6. Overview of character extraction pipeline

Clue board mask and extra cameras

Because all the clue boards had the same blue border colors, we found a mask that filtered only the clue board. To detect when a plate was close enough to be analyzed further, we monitored when the mask pixel count exceeded an experimental threshold, and when the greatest blob was a clean four-sided polygon (using cv2 contour finding and polygon approximations) and exceeding an area threshold, using cv2 contour area. The area threshold allowed us to filter the difference between a close plate and the backside of a farther plate, which had a lower area but higher pixel count. Rqt_graph is a useful tool for developing these thresholds.

A clear unobstructed view of some plates on the front camera was driving path dependent. To fix this problem and allow for greater independence between the driving and plate reading problem, we added two cameras in the .urdf file, mounted next to the front camera, facing 45 degrees to the left and right of the centerline. To reduce load on our simulation, we decreased the framerates to five hertz. Left and right images were stored in the background by a callback, and since the cameras were orthogonal, we didn't need to compare them and processed them in parallel at five hertz by a rospy timer callback.

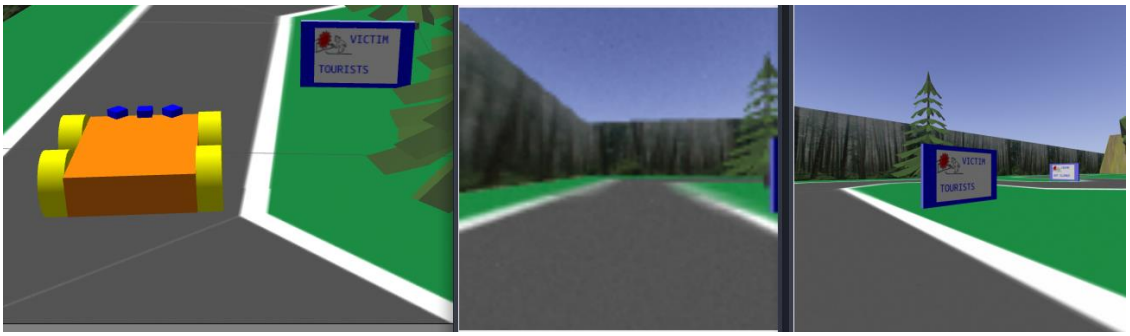

Figure 7. modified urdf, central camera not capturing plate, and right camera capturing full view

Solving plate detection

The approach to scanning each image for a satisfactory plate was a series of tests of increasing degrees of constraint, to ensure only high-quality character images went through to the CNN while minimizing unnecessary image processing. We were able to re-use the same series of tests

consecutively with different parameters for two purposes: first, to find a large enough clueboard by our blue mask and unskew it to a rectangular image, and next, to find and filter the gray rectangular background of the text within the blue clueboard and unskew it, ensuring consistency in character resolution, size, and orientation. As described above, these tests began with the blue mask to find a large four-sided polygon to analyze further.

An inverse perspective transform converted the skewed polygon (and its contents) into a rectangular image that could be passed into the same series of tests just described, but with different masks and thresholds. Having a second layer of tests and processing for the gray background also gave us assurance that we were scanning the front of the clue board.

Robust text extraction

The next step is extracting small but good quality character images to be read by a CNN. While the character placement was almost completely consistent, using hardcoded bounding boxes to isolate characters never worked perfectly, and often error would accumulate across a word, so that some characters would contain parts of the next character. Instead, we used cv2 contour finding to find each enclosed character, and when necessary padded its bounding box with extra pixels up to a 32 x 32 image.

However, if the text was too blurry then one contour would contain several characters, and if the text frayed, then fragments of characters were identified as contours and processed as full characters. Manually compiling fragmented contours was not an attractive solution to this problem, so we found a clean solution. If there was blurring without fraying, we'd recursively call the text extraction method with a stronger mask on the original image until the blurring stopped. If there was any fraying, we'd recursively call the text extraction method with a weaker mask until the fraying stopped, and manually deal with any resultant blurring by splicing contours that were too wide into the number of characters they contained, which was calculated by how wide they were relatively to an expected monospace character width.

Figure 8. Automatic weakening of text mask to prevent fraying

Lastly, to order the characters images into their respective words and find spaces that contour finding naturally can't pickup, I ordered the top left corners of each character into which section they belonged to (clue if in the top half of the original image, value if in the bottom half of the image), and then ordered horizontally, identifying space positions by the gap sizes between successive characters.

CNN Training

To generate an even character distribution, we overwrote the competition repository plate generation code to generate the clue board characters randomly. Also, we drew the correct clue value pairs from a csv file that the repository plate generation module wrote to. Sequentially teleporting the robot to the eight clue board locations with small random variation in position and orientation, we scanned all the characters and labelled them uniquely to write to a labelled training data directory. This enabled us to collect thousands of images in 10-20 minutes.

We used a CNN with 3 convolution layers each followed by max pooling, followed by flattening and a dense layer before the final layer with 36 outputs. The final layer used softmax activation, and all other layers used ReLU activation. We used categorical cross-entropy loss, the Adam optimizer with a learning rate of 2×10^{-3}, training for 30 epochs and taking the model with the lowest validation loss. See appendix for model architecture summary.

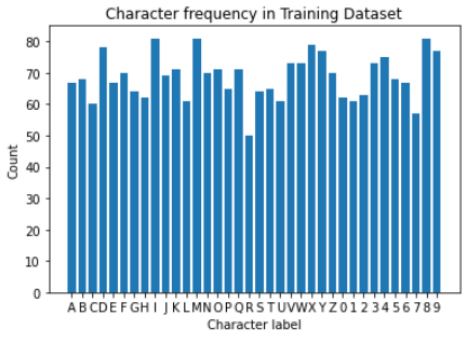

Figure 9. Histogram of character frequency in dataset

The confusion matrix is pictured in the appendix. To reduce model size, we quantized and pruned it according to a class reference, which reduced the size by a factor of 10, and gave a final inference time of around 0.1 millisecond, which was more than satisfactory for our purposes.

To ensure guesses were sent by their correct location, even in the event of a misread clue category, we found the clue category that yielded the minimum hamming distance (number of characters that differ index-wise between two strings) between the predicted clue category, and the set of possible clue categories.

Conclusion

Competition Performance

Driving worked well at the start, but an unresolved edge case with the truck led to the robot following the truck in loops until our preset 90s timer expired and stopped the competition. In the meantime, all signs were detected, though sign quality wasn't always perfect and 2 signs were predicted slightly incorrectly from similar-looking letters. Final score 1 point 90 seconds.

We shot a video of 4 consecutive runs (no cuts or edits) to show our average, expected performance. These performances would have placed us between 3rd-8th.
https://drive.google.com/file/d/1GMDxH6wexbVbnaEcWZrFjHmdYQoJBlu7/view?usp=sharing

Run #	Points	Time (s)	Timestamp
1	37	96	0:00-1:47
2	49	95	2:34-4:15
3	28	90*	4:43-5:56
4	50**	100	6:29-8:17
AVERAGE	41		

*cut short, but would have stopped at 90s

**forgot to include the respawn penalty in the video

One reason our competition performance did not match our expectations, or the video was that we ran the competition on Ezra's computer, which had different specifications and may have handled driving logic differently, whereas reliability testing was mostly done on Justin's computer. We did this because Justin's computer was unable to run the competition environment on competition day due to unanticipated circumstances; see Justin's logbook for details. This video was recorded on Justin's computer and shows expected performance had we not encountered this issue.

As you can see above, plate detection was less reliable than hoped for, as plates periodically were driven past. On integration in the days leading up to competition, we realized that thresholds for when a plate should be read to yield legible text depended on the clue board location because of the distance of our driving line to the plate. While we added some hardcoded adjustments to decrease the threshold on some plates, this wasn't properly tested and remained unreliable.

Methods attempted but not adopted

Both partners attempted imitation learning for the last section from different approaches, but used Justin's implementation, which worked first. Ezra tried controlling the robot with one continuous variable, kappa, the inverse of turning radius, following an NVIDIA paper on imitation learning for self-driving cars. A GUI for steering with implemented hotkeys for toggling data capture, upload, deletion was made with low resolution (96x72) gray-scaled images was developed, and after a preliminary model, intervention datapoints were easily generated. However, once our discrete action space model worked, we dropped the kappa one.

Figure 10. GUI for kappa controlled driving, data generation, and model intervention.

Things we would have done differently

Driving

Our biggest change would be pursuing imitation learning from the very start, as it was neither as time consuming nor as computationally intensive as we had expected, as our CNN for the final section demonstrates; lightweight driving CNNs can be trained in relatively little time. We originally thought that the state machine approach with line masking would be simple, but it turned out that the edge cases took a significant amount of time to deal with, and looking back it wasn't worth it; we were victims to the sunk cost fallacy. The rationale for artisanal methods (faster speed) did not end up holding up, as we appeared to run into ROS constraints that limited how fast our robot reacted to changes in the camera feed regardless of how quickly images could be processed. Even assuming a state machine-based approach, we should have spent more time spent on edge cases in section (i), making that as reliable as possible before completing the rest of the course, as the driving logic for everything else would be useless if the robot failed to pass the first section.

Plate Detection and Recognition

We would have worked towards greater transparency in the character recognition CNN test, and intentionally augmented and enriched my dataset to cater to areas where it struggled more. Had we done imitation learning I would have even implemented some script that passively collects character data while driving continuously throughout the track. This would have made for the most robust solution because the generated data would be completely real.

I think generally I regret not looking deeper into neural network training tuning parameters. I had essentially no practice going into the final CNN tuning, and especially for imitation learning I believe this was my stumbling block. The data problem is inherently highly dimensional, so it's overwhelming to know what to tackle.

References

NVIDIA imitation learning implementation: End-to-end learning for self-driving cars

Pruning and quantization guide: https://www.kaggle.com/code/sumn2u/pruning-and-quantization-in-keras#Fine-tune-pre-trained-model-with-pruning

Sanjiban Choudhury imitation learning lectures: https://www.youtube.com/watch?v=V38omEpfbjI&list=PLQZQ7N26C6ba2BDFVULmmBYC80cX6pNjZ

Appendices

Sample ChatGPT conversations

The main ways we used ChatGPT were for simple pythonic implementations for data wrangling, debugging issues that came up, and bouncing ideas off for sanity checks and input. Here is one such example that guided our decision process for plate detection by blue mask pixel count. For more examples, see our logbooks. https://chatgpt.com/share/691a6fa7-0f94-8009-a27d-5eac7e092a6a

CNN Architecture Summaries

Section (iv) driving CNN

Layer (type)	Output Shape	Param #
sequential_11 (Sequential)	(None, 160, 320, 1)	0
conv2d_35 (Conv2D)	(None, 80, 160, 16)	160
max_pooling2d_27 (MaxPooling2D)	(None, 40, 80, 16)	0
batch_normalization_26 (BatchNormalization)	(None, 40, 80, 16)	64
dropout_32 (Dropout)	(None, 40, 80, 16)	0
conv2d_36 (Conv2D)	(None, 40, 80, 32)	4,640
max_pooling2d_28 (MaxPooling2D)	(None, 20, 40, 32)	0
batch_normalization_27 (BatchNormalization)	(None, 20, 40, 32)	128
dropout_33 (Dropout)	(None, 20, 40, 32)	0
conv2d_37 (Conv2D)	(None, 20, 40, 64)	18,496
max_pooling2d_29 (MaxPooling2D)	(None, 10, 20, 64)	0
batch_normalization_28 (BatchNormalization)	(None, 10, 20, 64)	256
dropout_34 (Dropout)	(None, 10, 20, 64)	0

conv2d_38 (Conv2D)	(None, 10, 20, 128)	73,856
batch_normalization_29 (BatchNormalization)	(None, 10, 20, 128)	512
dropout_35 (Dropout)	(None, 10, 20, 128)	0
global_average_pooling2d_7 (GlobalAveragePooling2D)	(None, 128)	0
dense_20 (Dense)	(None, 128)	16,512
dense_21 (Dense)	(None, 5)	645

Total params: 115,269 (450.27 KB)

Trainable params: 114,789 (448.39 KB)

Non-trainable params: 480 (1.88 KB)

Character Recognition CNN

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 32, 32, 8)         0
 max_pooling2d (MaxPooling2D)  (None, 16, 16, 8)       0
 conv2d_1 (Conv2D)           (None, 16, 16, 16)        1168
 max_pooling2d_1 (MaxPooling2D)(None, 8, 8, 16)        0
 conv2d_2 (Conv2D)           (None, 8, 8, 32)          4640
 max_pooling2d_2 (MaxPooling2D)(None, 4, 4, 32)        0
 flatten (Flatten)           (None, 512)               0
 dropout (Dropout)           (None, 512)               0
 dense (Dense)               (None, 128)               65664
 dense_1 (Dense)             (None, 36)                4644
=================================================================
Total params: 76,196
Trainable params: 76,196
Non-trainable params: 0
```

Interesting plots and data examples

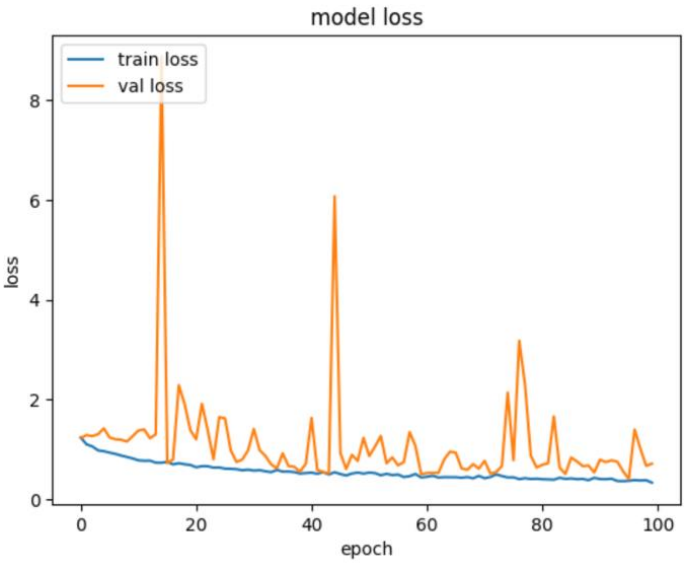

Figure 11. Driving CNN train and validation loss.

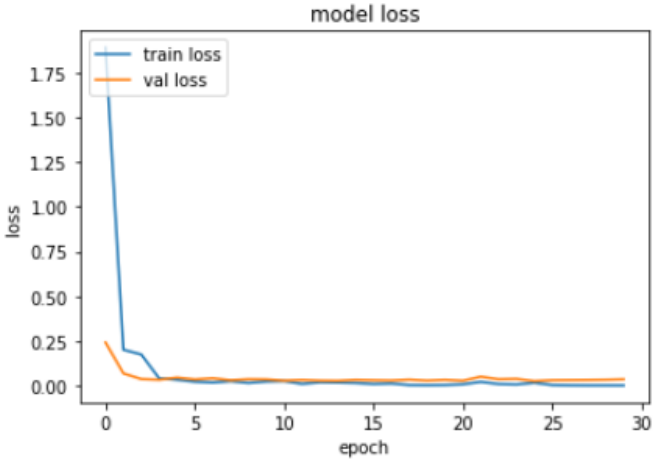

Figure 12. Character recognition CNN train and validation loss (unnecessary number of epochs, but training took seconds)

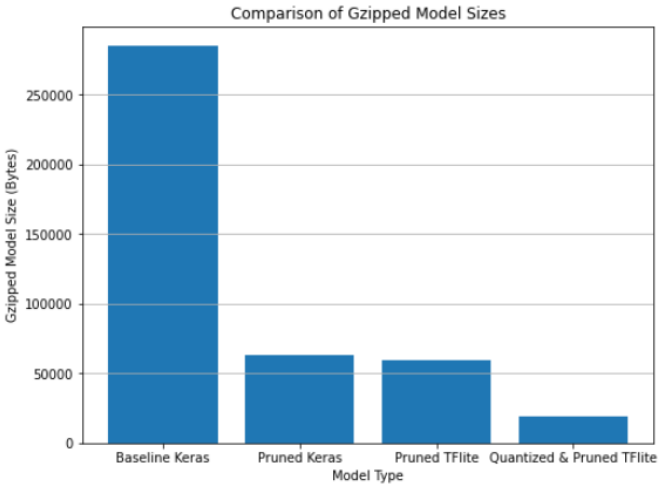

Figure 13. Quantizing and pruning character CNN size decreases.